Estructuras de Datos

Listas Enlazadas, Pilas y Colas

Dr. Pablo Ormeño-Arriagada

6 de abril de 2026

Contenidos

- Listas enlazadas
- Listas doblemente enlazadas
- Pilas (Stacks)
- Colas (Queues)

¿Qué son?

Definición

Una lista enlazada es una estructura dinámica donde cada nodo apunta al siguiente.

Lista Enlazada



Ventajas y Desventajas

Ventajas

- Inserciones eficientes
- Eliminaciones rápidas
- Tamaño dinámico

Desventajas

- Acceso secuencial
- Uso extra de memoria

Estructura de Nodo

```
struct Nodo {
    int dato;
    struct Nodo *siguiente;
};
```

Inserción al inicio

```
void insertarAlPrincipio(Nodo **cabeza, int valor) {
    Nodo *nuevoNodo = crearNodo(valor);
    nuevoNodo->siguiente = *cabeza;
    *cabeza = nuevoNodo;
}
```

Impresión de Lista

```c
void imprimirLista(Nodo *cabeza) {
    Nodo *actual = cabeza;
    while (actual != NULL) {
        printf("%d -> ", actual->dato);
        actual = actual->siguiente;
    }
    printf("NULL\\n");
}
```

Definición

Cada nodo tiene referencia al siguiente y al anterior.

Estructura

```
struct Nodo {
    int dato;
    struct Nodo *anterior;
    struct Nodo *siguiente;
};
```

Eliminación

```
void borrarPrimerNodo(Nodo **cabeza) {
    if (*cabeza == NULL) return;

    Nodo *temporal = *cabeza;
    *cabeza = (*cabeza)->siguiente;

    if (*cabeza != NULL)
        (*cabeza)->anterior = NULL;

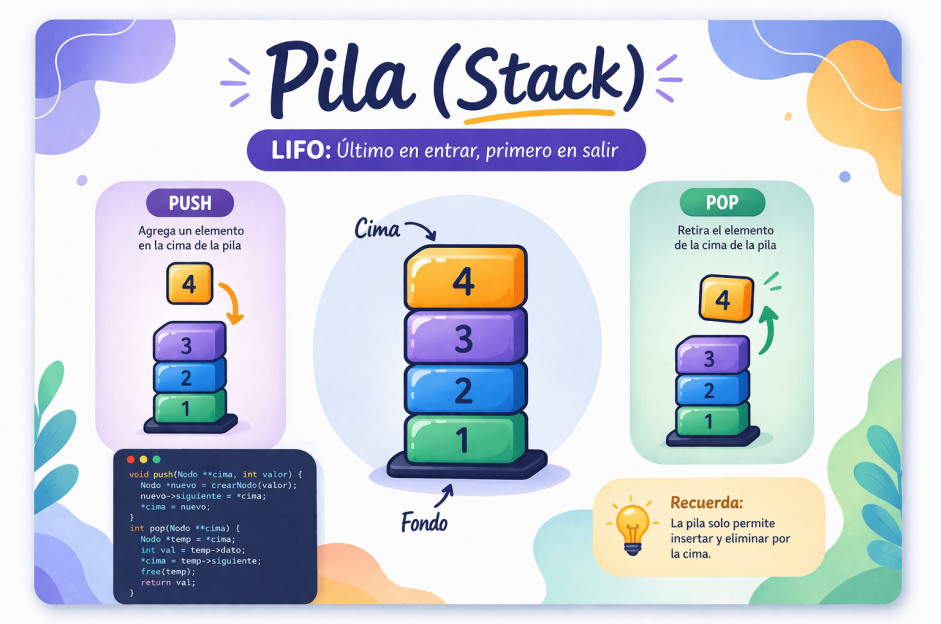
    free(temporal);
}
```

¿Qué es una pila?

LIFO

Último en entrar, primero en salir.

Pila (LIFO)


Pila (Stack)
LIFO: Último en entrar, primero en salir
PUSH
Agrega un elemento en la cima de la pila
4
3
2
1
Cima
4
3
2
1
Fondo
POP
Retira el elemento de la cima de la pila
4
3
2
1
Recuerda:
La pila solo permite insertar y eliminar por la cima.
void push(Nodo **cima, int valor) {
Nodo *nuevo = crearNodo(valor);
nuevo->siguiente = *cima;
*cima = nuevo;
}
int pop(Nodo **cima) {
Nodo *temp = *cima;
int val = temp->dato;
*cima = temp->siguiente;
free(temp);
return val;
}

Operaciones

- push (insertar)
- pop (eliminar)

Push

```
void push(NodoPila **cima, int valor) {
    NodoPila *nuevoNodo = crearNodo(valor);
    nuevoNodo->siguiente = *cima;
    *cima = nuevoNodo;
}
```

Pop

```
int pop(NodoPila **cima) {
    if (*cima == NULL) return -1;

    NodoPila *temp = *cima;
    int valor = temp->dato;

    *cima = temp->siguiente;
    free(temp);

    return valor;
}
```

¿Qué es una cola?

FIFO

Primero en entrar, primero en salir.

Cola (FIFO)


Cola (Queue)
FIFO: Primero en entrar, primero en salir
ENQUEUE
Agrega un elemento al final de la cola
DEQUEUE
Retira el elemento al frente de la cola
Frente
Final
6
6
1
2
3
4
5
1
1
void enquee(Nodo *ffrente, Nodo **valor) {
Nodo *nuevo = crearNodo(valor);
if (fin ke = NULL)
*frente = *fin = nuevo;
else {
ffin =>siguiente = nuevo;
*fin = nuevo;
}
}
Regnesa en entrarao
Tip:
En una cola, el primer elemento en entra es el primero en salir.

Operaciones

- enqueue (insertar)
- dequeue (eliminar)

Enqueue

```
void enqueue(NodoCola **frente, NodoCola **fin, int
    valor) {
     NodoCola *nuevoNodo = crearNodo(valor);

     if (*fin == NULL) {
          *frente = *fin = nuevoNodo;
     } else {
          (*fin)->siguiente = nuevoNodo;
          *fin = nuevoNodo;
     }
}
```

Dequeue

```
int dequeue(NodoCola **frente) {
    if (*frente == NULL) return -1;

    NodoCola *temp = *frente;
    int valor = temp->dato;

    *frente = temp->siguiente;
    free(temp);

    return valor;
}
```

Pilas vs Colas

Pila	**Cola**
LIFO	FIFO
push/pop	enqueue/dequeue

Conclusiones

- Listas permiten estructuras dinámicas
- Pilas manejan estados
- Colas gestionan procesos
- Estructuras fundamentales en programación

Fin de la clase